Дисциплина «API-технологии»

Методические указания к практической работе № 1

Анализ API-сценариев бизнес-системы с элементами ИИ: потребители, ресурсы, данные, контекстная схема

1. Для чего нужна эта работа

Прежде чем писать первую строку кода API, важно научиться видеть интерфейс глазами архитектора, а не программиста. API — это не просто набор функций, которые отвечают на запросы. Это контракт: соглашение о том, кто, зачем и в каком виде будет обращаться к системе, какие данные система готова отдавать и как она вписывается в общую картину бизнес-процесса. Если не продумать этот контракт заранее, любая последующая реализация — даже технически безупречная — окажется бесполезной, потому что не будет соответствовать реальным потребностям тех, кто ею пользуется.

Именно поэтому первая практическая работа курса не связана с написанием кода. Её задача — научить вас анализировать бизнес-сценарий и переводить его на язык API ещё до того, как появится хотя бы один эндпоинт. Результаты этой работы станут основой для проектирования REST API на следующем занятии, поэтому отнеситесь к ней как к фундаменту, а не как к формальному упражнению.

2. Что вы должны получить в результате

По итогам занятия у вас должны быть готовы два артефакта: контекстная схема системы и таблица интерфейсов. Контекстная схема — это простая визуальная диаграмма, показывающая, из каких компонентов состоит бизнес-система и как эти компоненты обмениваются данными через API. Таблица интерфейсов — это текстовое описание будущих точек взаимодействия: кто их вызывает, зачем и что получает в ответ. На этом этапе вас не должны интересовать технические детали вроде HTTP-методов

или JSON-схем — до них дойдёт очередь на следующих занятиях. Сейчас важно мышление на уровне ролей, данных и связей.

3. Как выполнять работу

Шаг 1. Выберите бизнес-сценарий.

Возьмите один из сквозных сценариев курса и работайте с ним на протяжении всей практики. Подходят следующие варианты: сервис прогнозирования спроса для розничной сети, сервис прогнозирования урожайности для агропредприятия, система кредитного скоринга для аграрного бизнеса или интеграционный сервис, связывающий ERP, CRM и BI-системы компании. Выбранный сценарий стоит зафиксировать письменно в одном-двух предложениях: что делает система, для какой отрасли и какую бизнес-задачу решает.

Шаг 2. Определите, кто будет пользоваться вашим API.

Подумайте, кто может обращаться к системе. Здесь стоит разделять потребителей на внешних и внутренних. Внешние — это, например, мобильное приложение клиента, сайт партнёра или сторонняя информационная система, которая интегрируется с вашим сервисом. Внутренние — это компоненты самой системы: другой микросервис, ETL-процесс, который регулярно загружает данные, или модуль машинного обучения, который выполняет инференс. Постарайтесь не ограничиваться одним очевидным потребителем — обычно у бизнес-системы их несколько, и они предъявляют разные требования к данным.

Шаг 3. Опишите ресурсы и данные.

Для каждого потребителя определите, какие данные ему нужны от системы. Это могут быть справочники (например, список культур или клиентов), исторические данные продаж, результаты прогноза, готовые отчёты или события — уведомления о том, что что-то произошло (появился новый заказ, обновился прогноз, завершилась загрузка данных). Постарайтесь сформулировать эти ресурсы как существительные, а не как функции: не

«посчитать прогноз», а «прогноз спроса на следующий месяц» как результат, который кто-то запрашивает.

Шаг 4. Найдите место API в жизненном цикле системы.

Любая бизнес-система с элементами ИИ проходит через несколько стадий: сбор данных, их обработка, обучение модели, инференс (применение модели к новым данным), мониторинг работы и, в конце концов, обновление или замена модели. Пройдитесь по своему сценарию и отметьте, на каком из этих этапов вашей системе потребуется API — например, API может понадобиться, чтобы забрать новые данные для переобучения, отдать результат инференса внешнему приложению или сообщить об ошибке в процессе мониторинга. Не обязательно, что API нужен на каждом этапе: важно показать, что вы понимаете, где именно он появляется и почему.

Шаг 5. Соберите контекстную схему.

На основе предыдущих шагов нарисуйте схему уровня «контекст»: в центре — ваша система, вокруг — все определённые вами потребители и внешние системы, а стрелки между ними подписаны так, чтобы было понятно, какие данные передаются и в каком направлении. Схему можно нарисовать от руки, в любом графическом редакторе или с помощью специализированных инструментов вроде draw.io или Mermaid — формат не принципиален, важна читаемость и точность связей. Каждая стрелка на схеме должна соответствовать одной строке в таблице интерфейсов, которую вы будете заполнять на следующем шаге.

Шаг 6. Заполните таблицу интерфейсов.

Для каждой связи на схеме составьте короткое описание: кто инициирует обращение, какую информацию он хочет получить или передать, и какая часть системы будет отвечать за это взаимодействие. На этом этапе достаточно словесного описания — конкретные HTTP-методы, пути и структуры JSON вы спроектируете на практической работе № 2. Таблица должна быть настолько понятной, чтобы человек, не участвовавший в вашем анализе, мог по ней восстановить логику взаимодействия компонентов системы.

4. Защита работы

В конце занятия каждый студент кратко представляет свою схему и таблицу интерфейсов — на выступление отводится около пяти-семи минут. Будьте готовы объяснить, почему вы выбрали именно этих потребителей, почему ресурсы описаны именно так и на каком этапе жизненного цикла системы возникает необходимость в API. Вопросы преподавателя и однокурсников будут касаться именно обоснованности ваших решений, а не технической реализации — она пока не требуется.

5. На что обратить внимание

Главная ошибка, которую допускают на этом занятии — слишком быстрый переход к техническим деталям. Если вы ловите себя на мысли «здесь нужен GET-запрос» или «это будет POST с телом в JSON» — остановитесь: сейчас не время об этом думать. Работа считается выполненной качественно, если по вашей схеме и таблице сразу понятно, кто и зачем взаимодействует с системой, даже без единой строчки кода. Это ровно то умение, ради которого существует данная практика, и именно она станет основой для проектирования полноценного REST API на следующем занятии.

Материал подготовлен в рамках дисциплины «API-технологии».